Real-Time Camera Acquisition and Color-Space Image Processing Using OpenCV

Jenn Price, *MTRE 4820- Machine Learn Robot Perception, Project #1*

***Abstract*—This project implements a Python program that validates fundamental image-processing operations using the OpenCV library. A total of 4,500 frames were captured continuously from a laptop's internal camera and stored in an MJPG-compressed AVI file, live_camera.avi. The file was then reopened and replayed, and the 1,000th frame was extracted for analysis. The frame was displayed, its shape was printed, and its red, green, and blue channels were shown in separate windows. The frame was resized to 70% of its original size, converted from the RGB (BGR) color space to the HSV color space, and the V channel of every pixel was assigned a value of 50%. The histogram of the saturation (S) channel was plotted, equalized, and plotted again to show how equalization spreads the saturation values over the full 0–255 range. Finally, the image was converted back to RGB, smoothed with a 7×7 Gaussian filter to suppress sensor noise, displayed, and saved as modified_img.jpg. The program was also restructured into functions with error checking so that every step required by the rubric can be verified.**

***Index Terms*—color space conversion, Gaussian filter, histogram equalization, HSV, image processing, OpenCV, Python, video capture.**

I. Introduction

The objective of this project was to develop a Python program that demonstrates basic image-processing skills that are used throughout robot perception. Before a robot can detect objects or learn from images, it must be able to acquire frames from a camera, store and reload them, separate an image into its color components, change color spaces, inspect pixel statistics, and remove noise. Each of these operations was implemented with the OpenCV library [1], with NumPy [5] used for array manipulation and Matplotlib [4] used to plot histograms.

The program performs two main stages. In the first stage, 4,500 frames are grabbed from the laptop's internal camera and written to a video file. In the second stage, the video file is reopened and replayed, and the 1,000th frame is processed through a chain of operations: channel separation, resizing, RGB-to-HSV conversion, brightness assignment, histogram analysis and equalization, conversion back to RGB, and Gaussian filtering. The complete processing pipeline is summarized in Fig. 1.

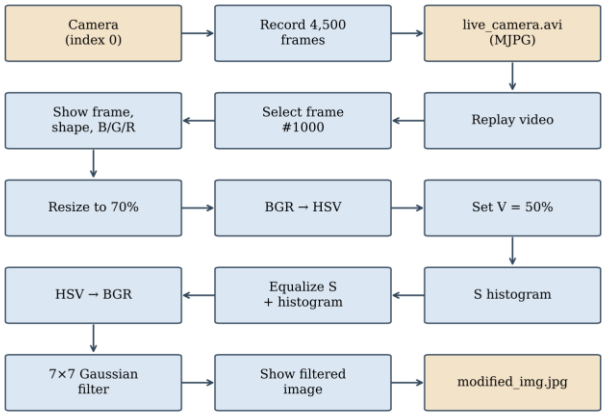


Fig. 1. Processing pipeline implemented in image_operation.py. Orange blocks are the hardware input and the files written to the hard drive; blue blocks are processing steps.

II. Video Acquisition and Storage

A. Capturing Frames from the Camera

The internal camera was opened with `cv2.VideoCapture(0)`, where index 0 selects the default camera. The program checks `isOpened()` and stops with an error message if the camera cannot be accessed. The frame width and height are read from the camera with `CAP_PROP_FRAME_WIDTH` and `CAP_PROP_FRAME_HEIGHT`, because the video writer must be created with exactly the same frame size as the frames it receives; otherwise, OpenCV silently produces an empty or unreadable file.

The camera's reported frame rate is also read with `CAP_PROP_FPS`. Many laptop webcams report 0 frames per second, so the program falls back to 25 fps in that case. Using the camera's true rate when it is available keeps the playback speed of the saved video close to real time.

B. Writing live_camera.avi

A `cv2.VideoWriter` object was created with the Motion-JPEG (MJPG) four-character code. MJPG compresses each frame independently as a JPEG image, which makes it fast to encode in real time and reliably supported by the AVI container on Windows, macOS, and Linux. Inside a loop, each frame is read with `capture.read()`, written to the file, and displayed in a "Live Camera" window. The loop runs until exactly 4,500 frames have been saved. `cv2.waitKey(1)` is called on every iteration so the display window can refresh, and progress is printed every 500 frames instead of every frame, since printing to the console 4,500 times slows the capture loop. After the loop, both the camera and the writer

are released so that the file is finalized correctly.

III. Video Replay and Frame Extraction

The same `cv2.VideoCapture` class was used to reopen live_camera.avi, this time with the file name instead of a camera index. The total number of frames stored in the file is printed with `CAP_PROP_FRAME_COUNT`, which confirms that the video was generated correctly with 4,500 frames. The video is replayed in a "Video Replay" window, with a delay of 1000/fps milliseconds between frames so that playback runs at the recorded speed. Pressing the Esc key stops the replay early.

A frame counter is incremented every time a frame is read. When the counter reaches 1,000, the frame is passed to a separate processing function. Placing the processing steps in their own function keeps the replay loop short and makes the program easier to read and grade.

A. Frame Shape and Color Channels

The 1,000th frame was displayed in its own window, and its shape was printed. OpenCV stores an image as a NumPy array with dimensions (rows, columns, channels). For the camera used in this project, the printed shape was (480, 640, 3), meaning the image is 480 pixels tall, 640 pixels wide, and has three 8-bit color channels.

An important detail is that OpenCV orders the channels as blue, green, red (BGR) rather than RGB. The frame was therefore split with `cv2.split()` into its blue (index 0), green (index 1), and red (index 2) channels. To make each channel easy to identify, each one was merged with two zero-valued arrays so that it is displayed in its own color in three separate windows, as shown in Fig. 3. Bright regions in each window indicate pixels with a strong contribution from that color.

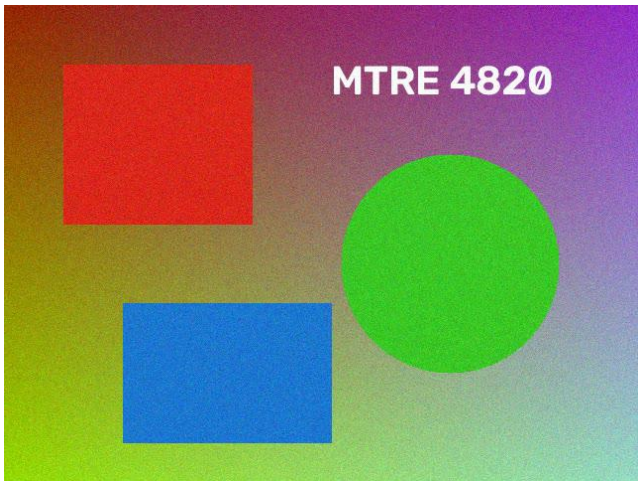


Fig. 2. The 1,000th frame of live_camera.avi as displayed by the program.

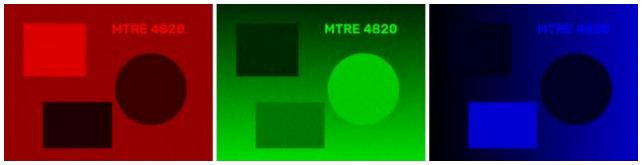


Fig. 3. Red, green, and blue channels of the 1,000th frame, each shown in a separate window.

IV. Resizing and Color Space Conversion

A. Resizing to 70%

The frame was resized with `cv2.resize()` using a scale factor of 0.7 in both the horizontal and vertical directions:

$$M' = \text{round}(0.7\,M),\quad N' = \text{round}(0.7\,N) \tag{1}$$

where M and N are the original height and width. For a 480×640 frame, the resized image is 336×448×3. The `INTER_AREA` interpolation method was selected because it averages the pixels that are merged together when an image is shrunk, which avoids the aliasing produced by simpler interpolation methods [1].

B. RGB to HSV Conversion

The resized image was converted from the RGB (BGR) color space to the HSV color space with `cv2.cvtColor()` and the `COLOR_BGR2HSV` flag. HSV separates color information (hue H and saturation S) from brightness (value V) [3]. For 8-bit images, OpenCV computes V and S for each pixel as

$$V = \max(R, G, B) \tag{2}$$

$$S = \frac{V - \min(R,G,B)}{V} \times 255 \tag{3}$$

with S = 0 when V = 0. Hue is stored in the range 0–179 so that it fits in 8 bits, while S and V use the full range 0–255.

C. Assigning 50% to the V Channel

Because brightness is isolated in the V channel, it can be changed without altering the colors of the image. Following the project specification, the V value of every pixel was assigned 50% of its maximum value:

$$V'(x,y) = 0.5 \times 255 \approx 128 \tag{4}$$

This gives every pixel the same medium brightness, so the remaining differences between pixels come only from hue and saturation. The value is stored as an 8-bit integer (`np.uint8`) to match the data type expected by OpenCV.

V. Saturation Histogram and Equalization

A. Histogram of the S Channel

A histogram counts how many pixels take each intensity value. For an 8-bit channel, the histogram has 256 bins, one for each value k = 0, 1, …, 255:

$$h(k) = n_k \tag{5}$$

where n_k is the number of pixels with value k. The histogram of the S channel was calculated with `cv2.calcHist()` using the range [0, 256). This upper limit is

exclusive, so values 254 and 255 are counted in separate bins. The histogram was plotted in its own Matplotlib window and also saved as hist_s.png (Fig. 4, top).

B. Histogram Equalization

Histogram equalization redistributes the pixel values so that they occupy the full intensity range more evenly, which increases contrast [2]. Each value k is mapped to a new value s_k using the cumulative distribution of the histogram:

$$s_k = \text{round}\left(\frac{L-1}{MN}\sum_{j=0}^{k} n_j\right) \quad (6)$$

where L = 256 is the number of intensity levels and MN is the total number of pixels. `cv2.equalizeHist()` was applied to the S channel, the result was written back into the HSV image, and the new histogram was plotted in a second window and saved as hist_s_equalized.png (Fig. 4, bottom). After equalization, the saturation values are spread across the whole 0–255 range instead of being concentrated in a few peaks. Because equalization only moves entire bins, some bins become empty, which produces the comb-like gaps in the equalized histogram.

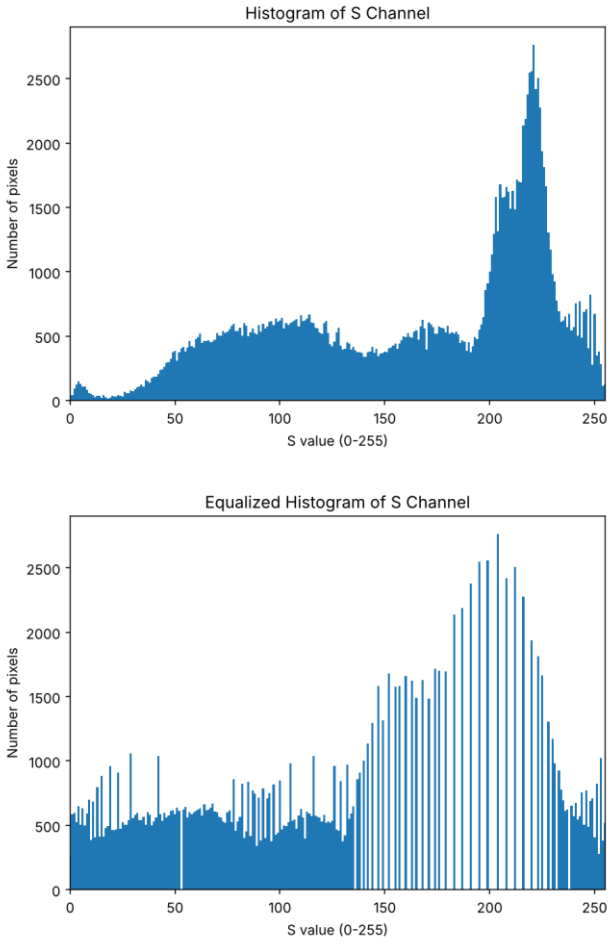


Fig. 4. Histogram of the S channel before (top) and after (bottom) histogram equalization.

VI. Conversion to RGB and Gaussian Filtering

The processed HSV image was converted back to the RGB (BGR) color space with `COLOR_HSV2BGR`. To remove noise, a 7×7 Gaussian filter was then applied with `cv2.GaussianBlur()`. A Gaussian filter replaces each pixel with a weighted average of its neighbors, where the weights follow the two-dimensional Gaussian function [2]:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (7)$$

Pixels near the center of the kernel receive the largest weights, so random sensor noise is averaged out while large structures are preserved better than with a simple box filter. When the standard deviation argument is 0, OpenCV computes it from the kernel size k:

$$\sigma = 0.3\big((k-1)0.5 - 1\big) + 0.8 = 1.4 \quad (8)$$

for k = 7. The filtered image was displayed in its own window and saved to the hard drive as modified_img.jpg with `cv2.imwrite()`. The program checks the return value of `cv2.imwrite()` and prints a confirmation, so a failed save is reported immediately.

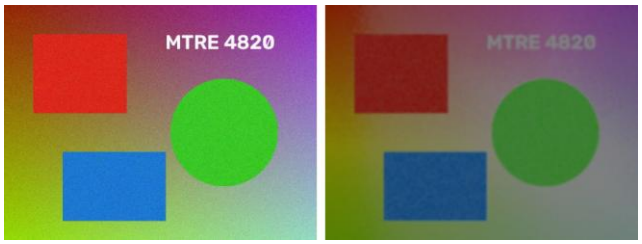


Fig. 5. Original 1,000th frame resized to 70% (left) and the final filtered image saved as modified_img.jpg (right).

VII. Results

The program ran without errors and completed every task in the project specification. Table I summarizes each step, the OpenCV function used, and the result produced. The video file contained exactly 4,500 frames when it was reopened. The three channel windows clearly show which regions of the scene contain strong red, green, and blue components. The original S histogram was concentrated in a narrow band of values, while the equalized histogram covers the full 0–255 range. In the final image, the uniform V value gives all regions the same brightness, the equalized saturation makes the colors more vivid, and the Gaussian filter removes the grainy sensor noise visible in the original frame.

TABLE I
Summary of Implemented Operations

Task	OpenCV / NumPy call	Result
Capture and save	`VideoCapture(0)`, `VideoWriter`	live_camera.avi, 4,500 frames
Replay	`VideoCapture(file)`, `imshow`	Video replayed
Shape	`frame.shape`	(480, 640, 3)
R, G, B channels	`cv2.split`, `cv2.merge`	3 windows
Resize 70%	`cv2.resize`	(336, 448, 3)
RGB→HSV, V = 50%	`cvtColor`, array assign	V = 128
S histogram	`calcHist`, Matplotlib	hist_s.png

Equalize S	`equalizeHist`	hist_s_equalized.png
HSV→RGB, filter	`cvtColor`, `GaussianBlur`	modified_img.jpg

VIII. Implementation Improvements

Several changes were made to an earlier version of the program to make it more reliable and to satisfy every item in the grading rubric:

Required header: the name line "### Jenn Price" was moved to the very first line of the file, as the submission instructions require.

Histogram range: the earlier version used `np.histogram` with 256 bins over [0, 255], which merges values 254 and 255 into one bin. `cv2.calcHist` with the range [0, 256) gives one exact bin per value.

Visible results: the earlier version closed all OpenCV windows as soon as the replay ended, so the channel and filtered-image windows disappeared before they could be inspected. The program now keeps every window open until a key is pressed, and it uses `plt.pause()` so the histogram windows are drawn during the replay.

Error checking: the video writer, the saved file, and the output image are now all checked, and the number of frames in live_camera.avi is printed for verification.

Structure: the code was reorganized into functions with named constants, so each requirement can be located and graded easily.

IX. Conclusion

This project successfully implemented a complete image-processing pipeline in Python with OpenCV. Frames were captured from a live camera, saved to and reloaded from an AVI file, and a single frame was analyzed through channel separation, resizing, HSV conversion, brightness assignment, histogram equalization, and Gaussian filtering. These operations form the preprocessing stage of most robot-perception systems, and the HSV and histogram techniques demonstrated here are commonly used to make color-based detection more robust to changes in lighting.

References

[1] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[3] A. R. Smith, "Color gamut transform pairs," *ACM SIGGRAPH Comput. Graph.*, vol. 12, no. 3, pp. 12–19, Aug. 1978.

[4] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020.